



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Inurreta Brito Gustavo Enrique

Introducción

En esta actividad se analiza la importancia del control de versiones durante el desarrollo de software y los problemas que pueden surgir cuando un equipo trabaja sin una herramienta adecuada para administrar los cambios del proyecto además, se explica qué es un sistema de control de versiones, cuáles son sus principales ventajas y cómo herramientas como Git ayudan a organizar el trabajo entre varios programadores mediante el uso de ramas y un historial de cambios.

Parte 1. Análisis del Problema

Explica con tus propias palabras:

1.¿Cuáles fueron los principales errores cometidos por el equipo durante el desarrollo?

Cada desarrollador descargaba una copia del sistema, realizaba modificaciones y posteriormente reemplazaban la carpeta completa con su versión de cada integrante con esto los cambios de cada programador fueron sustituidos por los archivos de la siguiente persona que remplazaron los archivos

2.¿Por qué trabajar únicamente compartiendo carpetas genera tantos problemas?

por la sobreescritura de archivos cuando un programador mueve un archivo y resube toda la carpeta completa del proyecto puede eliminar archivos modificados de otra persona dejando lo archivos en blanco que no modificó

3.¿Qué consecuencias podría tener esta forma de trabajo en proyectos grandes?

La mala comunicación y los descuidos de los programadores o el equipo puede llevar a un retraso en la entrega del los avances

Parte 2.- Importancia del Control de Versiones

Describe:

¿Qué es un sistema de control de versiones?

Es un gestor de cambios o actualizaciones dentro de un proyecto

¿Cuál es su principal objetivo?

llevar un control de las actividades, cambios o correcciones realizadas por el equipo de desarrollo

¿Qué ventajas ofrece frente al manejo tradicional de archivos?

La organización y separación de tareas ayuda a no interferir en modificaciones o tareas de otras personas

Parte 3. Propuesta de organización

Supón que ahora el equipo utilizará Git para administrar el proyecto.

Propón una estrategia de trabajo indicando:

- **¿Qué rama utilizarías para la versión estable del sistema?**

si se refiere a la rama donde estará la última versión del proyecto sería la rama main

- **¿Qué rama utilizarías para integrar el trabajo del equipo?**

una rama llamada pruebas o testeo

- **¿Cada desarrollador debería trabajar directamente sobre la rama principal? Explica tu respuesta.**

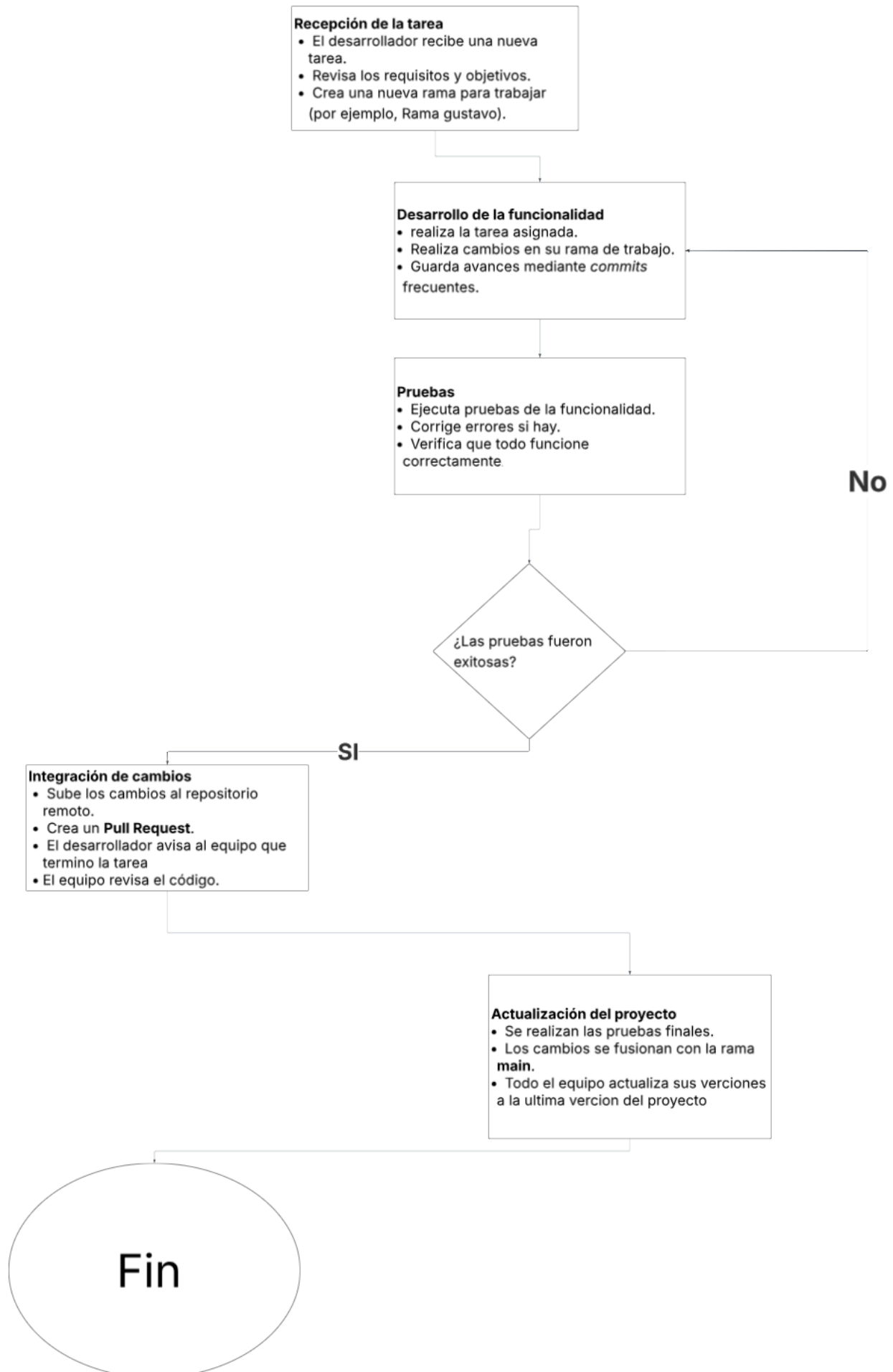
no de ´preferencia no por que al trabajar con la rama principal puede o muy seguramente provocará un conflicto con la rama (testeo/pruebas) cuando el proyecto esté listo para la carga a la rama principal (main)

- **¿Cuándo deberían integrarse los cambios al proyecto?**

cuando se terminan las modificaciones y se probó que no hay ningún error ahí ya podemos cargar los cambios y comprobemos que no hay conflictos con archivos ya cargados

Parte 4. Flujo de Trabajo

Elabora un diagrama (puede ser de bloques o de flujo) que represente el proceso que seguiría un desarrollador desde que recibe una tarea hasta que su código forma parte del proyecto principal.



Parte 5. Análisis de situaciones

Responde las siguientes preguntas.

1.¿Qué ocurriría si dos desarrolladores modifican el mismo archivo simultáneamente?

ocurriría un conflicto y el líder del equipo tendría que escoger qué archivo funciona más eficientemente de los 2 y descartar el otro

2.¿Por qué es importante conservar un historial de cambios en un proyecto?

por si ocurre conflictos con las versiones o fallos muy graves poder regresar a una versión estable del proyecto envés de empezar de cero o poder saber quien accidentalmente o a propósito provocó la falla o el error

3.¿Qué beneficios aporta trabajar mediante ramas independientes?

cada persona podrá trabajar sus tareas sin perjudicar a nadie y si hacen sus tareas solo las que se les pide no habrá conflicto entre las ramas y es una manera de trabajar más organizada y limpia

4.¿Cómo ayuda el control de versiones cuando aparece un error después de una actualización?

por si ocurre conflictos con las versiones o fallos muy graves poder regresar a una versión estable del proyecto envés de empezar de cero o poder saber quien accidentalmente o a propósito provocó la falla o el error

5.¿Por qué el control de versiones es indispensable en proyectos desarrollados por varios programadores?

Porque permite llevar un historial de los cambios realizados durante el proyecto y ayuda a que varios programadores trabajen al mismo tiempo sin afectar el trabajo de los demás. También facilita corregir errores y recuperar versiones anteriores si es necesario.

Parte 6. Conclusión

Redacta una conclusión personal donde expliques:

- **Qué aprendiste sobre el control de versiones.**

Es importante para la gestión de un proyecto y el poder llevar un historial de lo que van haciendo cada miembro del equipo también ayuda a recuperar versiones anteriores y evitar la pérdida de información.

- **Por qué Git se ha convertido en un estándar de la industria.**

es indispensable para el trabajo en equipo de manera ordenada, la gestión de un proyecto, el control de las versiones y el almacenamiento del proyecto

- **Cómo influye una buena organización del código en el éxito de un proyecto.**

Influye porque hace que cada integrante del equipo pueda encontrar y modificar el código más fácilmente, evitando errores y mejorando el desarrollo del proyecto.